LangGraph, Agent Lifecycle and Multi-Agent Coordination

A practical guide to building AI agents that actually finish the job

What is inside

- Part 1 - LangChain and LangGraph: what each one is for and how they fit together
- Part 2 - The agent lifecycle, from the first user message to the final answer
- Part 3 - Multi-agent coordination patterns, plus a note on what MCP really means
- Part 4 - Putting it together, common mistakes and a learning path

Introduction: why this topic matters

A year or two ago, most people used large language models the same way: type a question, get an answer, move on. That is a single step. Real work rarely fits into a single step. If you ask an assistant to research a competitor, write a summary and email it to your team, something has to look things up, decide what matters, write, check and send. Each of those is a separate action, and some of them depend on what happened in the one before.

That is what an **agent** is: a program that uses a language model to decide what to do next, does it, looks at what happened and keeps going until the job is finished. Building one is easy for a demo and surprisingly hard for production. The agent forgets things, loops forever, calls the wrong tool or does something you never wanted it to do.

This guide walks through three ideas that help with exactly those problems. The first is the pair of frameworks **LangChain** and **LangGraph**. The second is the **agent lifecycle**, which is a way of thinking about what an agent does from start to finish. The third is **multi-agent coordination**, which covers how several agents can share the work. I have tried to explain each one in plain language, with small examples and diagrams, so you can read this once and come away able to explain it to someone else.

Part 1: LangChain and LangGraph

What LangChain is

LangChain is an open-source library that gives you ready-made building blocks for applications built on language models. Instead of writing the raw code to talk to every model provider, format prompts, parse answers and call outside services, you plug together components that already do those jobs.

The main building blocks are easy to remember if you think of them as a toolbox:

- **Models** - a common interface to chat models from many providers, so switching from one to another is usually a one-line change.
- **Prompts** - templates with blanks that you fill in at run time, so you are not gluing strings together by hand.
- **Tools** - ordinary functions (search the web, query a database, run code) that the model is allowed to request.
- **Retrievers and vector stores** - the pieces behind retrieval-augmented generation, where the model gets relevant documents before it answers.
- **Output parsers and structured output** - ways to make the model return clean JSON or typed objects instead of loose prose.

For straightforward jobs, such as answer a question using these documents, LangChain on its own is plenty.

What LangGraph is, and why it exists

The trouble starts when the job is not a straight line. Imagine an agent that writes code, runs it, sees an error, fixes the code and runs it again. That is a loop, and loops are awkward to express as a simple chain of steps. You also want to be able to pause the process so that a human can approve something, save progress so a crash does not wipe out an hour of work, and see exactly which path the agent took.

LangGraph was built for that. It is a library, from the same team as LangChain, that lets you describe your agent as a **graph**. Think of a flowchart that the computer can actually run. It has three core ideas:

- **State** - a shared piece of data (often a typed dictionary) that every step can read and update. Messages so far, tool results, a counter, a draft: all of it lives here.
- **Nodes** - the steps. Each node is a normal function that takes the current state and returns the changes it wants to make.
- **Edges** - the arrows that decide what runs next. They can be fixed, or *conditional*, meaning a function looks at the state and picks the next node.

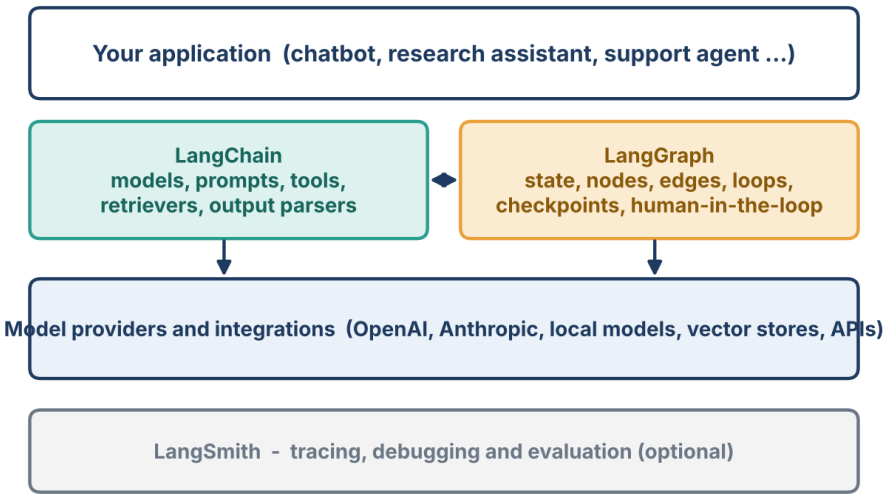


Figure 1. Where LangChain and LangGraph sit in a typical application. They are complementary, not competitors.

A fair way to put it: LangChain gives you the parts, and LangGraph gives you the control flow. Most real projects use both. You might use LangChain's chat model and tool wrappers inside nodes of a LangGraph workflow.

A tiny LangGraph example

The example below is deliberately small. It has one node that fills in an answer. The point is the shape: define the state, add nodes, connect them with edges, compile, run.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    question: str
    answer: str

def answer_node(state: State):
    # In a real app this is where you would call a model
    return {"answer": f"You asked: {state['question']}"}

builder = StateGraph(State)
builder.add_node("answer", answer_node)
builder.add_edge(START, "answer")
builder.add_edge("answer", END)

graph = builder.compile()
print(graph.invoke({"question": "What is LangGraph?"}))
```

The classic agent pattern adds a second node for tools and a conditional edge. After the model responds, the edge checks whether the model asked to use a tool. If it did, the flow goes to the tools node and then back to the model. If not, it ends. That back-and-forth is the loop that makes it an agent.

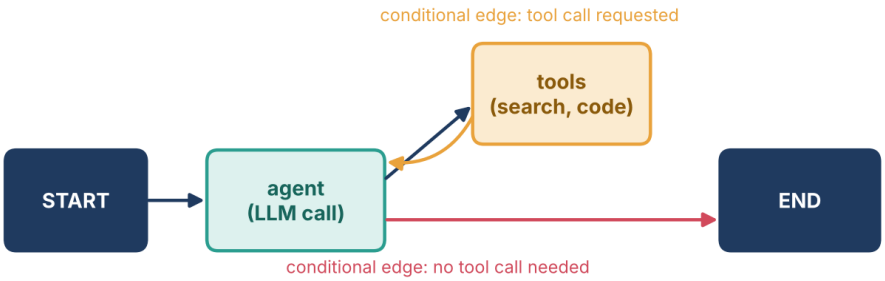


Figure 2. The standard tool-calling agent expressed as a graph. The loop between agent and tools is the heart of it.

Features worth knowing about

- **Persistence and checkpoints** - the graph can save its state after each step, which gives you memory across turns and the ability to resume after a failure.
- **Human-in-the-loop** - you can pause before a sensitive step, such as sending an email or spending money, and wait for a person to approve or edit.
- **Streaming** - you can show tokens and intermediate steps to the user as they happen, so the app does not feel frozen.

- **Time travel and debugging** - because states are saved, you can look back at an earlier step, change something and replay from there.
- **Subgraphs** - a whole graph can be used as a single node inside a bigger graph, which is how larger systems stay manageable.

> ***A practical note.*** *These libraries change quickly. Function names and recommended helpers get renamed between versions, so always check the current official documentation before you copy code from any guide, including this one.*

When to use which

Situation	Good fit	Why
Question answering over documents	LangChain alone	A simple retrieve-then-answer chain is enough
Agent that calls a few tools in a loop	LangGraph (with LangChain parts)	You need a loop and a clear stop condition
Workflow needing approval or long pauses	LangGraph	Checkpoints and interrupts are built in
Several specialist agents working together	LangGraph	Graphs model hand-offs and routing naturally
One quick model call	Neither	Call the provider directly; frameworks add weight

Part 2: The agent lifecycle

When people say lifecycle in software, they usually mean the stages something goes through from creation to retirement. For an agent there are two useful ways to look at it. The first is the **run-time loop**, which is what happens during a single task. The second is the **development lifecycle**, which is how an agent goes from idea to something you trust in production. Both matter, so I will cover each.

The run-time loop

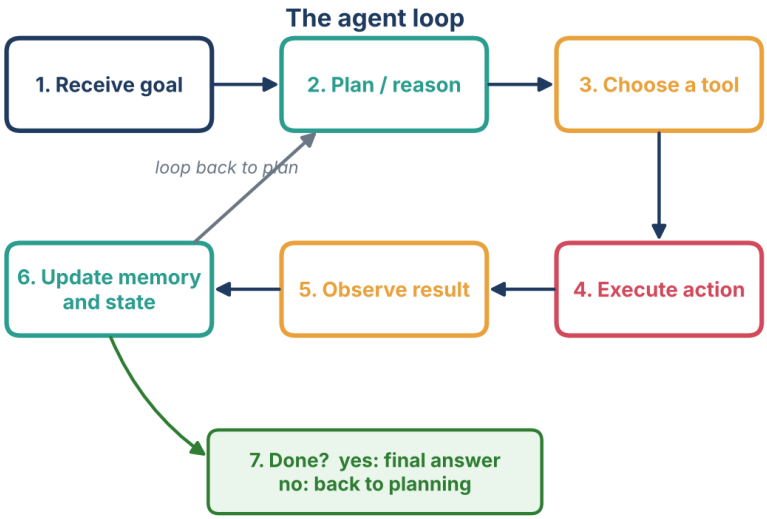


Figure 3. One pass through the loop. Hard tasks go around several times before the agent decides it is finished.

Here is what happens at each stage, in plain terms.

- **1. Receive the goal.** The user, or another system, gives the agent a task. Good agents also receive instructions about their role, limits and tone, usually in a system prompt.
- **2. Plan and reason.** The model looks at the goal and the current state and decides what to do next. Sometimes this is an explicit plan with numbered steps. Sometimes it is just choosing the next action.
- **3. Choose a tool.** If the agent needs information or needs to change something in the world, it picks a tool and fills in the arguments, for example *search(query=...)*.
- **4. Execute the action.** Your code, not the model, actually runs the tool. This is an important boundary: the model only *requests* actions, and your application decides whether to carry them out.
- **5. Observe the result.** The tool output is added to the conversation so the model can read it. This might be search results, a file, an error message or a database row.

- **6. Update memory and state.** The agent keeps what is useful. Short-term memory is the running conversation; long-term memory is information saved to a store so it can be recalled in later sessions.
- **7. Decide if it is done.** If the goal is met, the agent writes the final answer. If not, it loops back to planning with everything it has learned.

Why agents fail at this stage

Almost every failure I have seen comes from one of a few places. The agent loops without making progress because there is no limit on the number of steps. It picks the wrong tool because the tool descriptions are vague. It forgets an early instruction because the conversation grew too long. Or it trusts a tool result that was wrong. The remedies are mostly unglamorous: set a maximum step count, write clear tool descriptions, trim or summarise old messages, and validate results before acting on them.

The development lifecycle

Building a dependable agent is more like training a new employee than writing a normal program. You give it a job, watch how it behaves, correct it and slowly give it more responsibility. A sensible sequence looks like this:

Stage	What you do	What good looks like
Define	Write down the task, the users, what success means and what the agent must never do	A one-page spec that a colleague can read and understand
Design	Choose the graph structure, tools, memory approach and where humans must approve	A diagram with clear loops, stop conditions and permissions
Build	Implement nodes, tools and prompts; start with the simplest version that works	A basic agent that handles the happy path end to end
Test and evaluate	Create a set of realistic example tasks and score the results, including tricky ones	A repeatable test set and a number you can track over time
Deploy	Add logging, tracing, rate limits, cost limits and error handling	You can see what any run did, and a bad run cannot do serious damage
Monitor	Watch real usage, collect failures and feedback, and review costs	Problems are noticed by you before they are noticed by users
Improve	Fix the failures, add examples to your test set, update prompts and tools	Each release is measurably better than the last

The evaluation step is the one people skip, and it is the one that matters most. Without a test set you cannot tell whether a change made things better or worse; you are only going on a feeling. Tools such as LangSmith exist to trace each run and store examples for exactly this purpose, though you can build a simple version yourself with a spreadsheet and a script.

Safety and control throughout the lifecycle

- **Least privilege.** Give the agent only the tools and permissions the job needs. A research agent does not need permission to delete files.
- **Approval gates.** Require a human to confirm actions that are hard to undo: payments, sending messages to customers, changing records.
- **Budgets.** Set caps on steps, time and spending per run, so a confused agent stops by itself.
- **Logging.** Keep a record of every model call and tool call. When something goes wrong, the log is how you find out why.

Part 3: Multi-agent coordination patterns

One agent with a dozen tools can handle a surprising amount. But there comes a point where it becomes confused, because its instructions are long, its tool list is crowded and its context window is full of unrelated material. A common solution is to split the work among several smaller agents, each with a narrow job, fewer tools and a short, focused prompt. This is the idea behind multi-agent systems.

Splitting the work creates a new question: how do the agents talk to each other and decide who does what? The answer is a **coordination pattern**. Three of them cover most real cases.

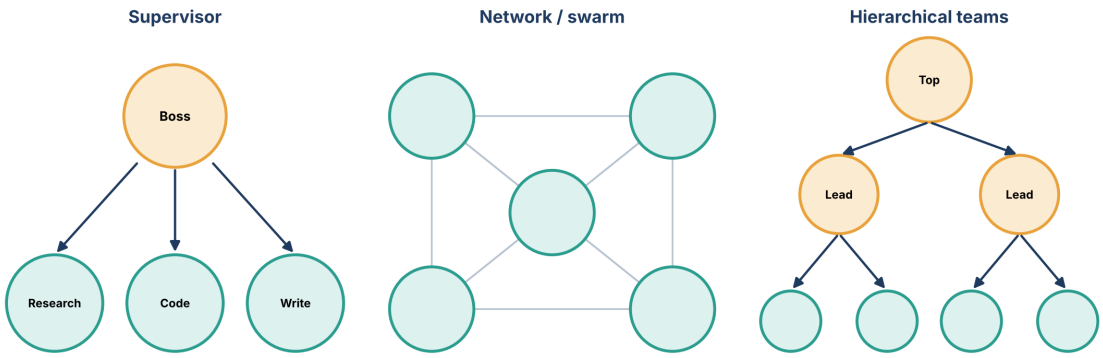


Figure 4. Three common ways to organise a group of agents.

Pattern 1: Supervisor

One agent, the supervisor, receives the task, decides which specialist should handle it, passes it on and collects the result. The specialists do not talk to each other; everything goes through the supervisor. This is the easiest pattern to understand and debug, because there is always one place where decisions are made.

Strengths: clear control, easy to trace, simple to add a new specialist. **Weaknesses:** the supervisor can become a bottleneck, and if its judgement is poor, the whole system suffers.

Pattern 2: Network (swarm or hand-off)

Here any agent can pass work directly to any other agent. A customer-support example would be a triage agent that hands a billing question to the billing agent, which then hands it to a refunds agent. There is no permanent boss; control moves along with the task.

Strengths: flexible, and well suited to conversations that change direction. **Weaknesses:** harder to predict. Agents can bounce a task back and forth, so you need clear rules about when to hand off and a limit on how many hand-offs are allowed.

Pattern 3: Hierarchical teams

This extends the supervisor idea. A top-level agent manages team leads, and each lead manages its own group of workers. A research team might have one lead for gathering sources and

another for writing, each with its own helpers. It mirrors how companies are organised.

Strengths: scales to large, complicated tasks, and each team can be built and tested separately.
Weaknesses: more moving parts, more cost, and more places for messages to get lost.

Comparing the patterns

	Supervisor	Network	Hierarchical
Control	Central	Distributed	Layered
Predictability	High	Lower	Medium to high
Setup effort	Low	Medium	High
Best for	Clear task routing	Fluid conversations	Large multi-part projects
Main risk	Bottleneck	Endless hand-offs	Complexity and cost

How agents share information

Whatever the pattern, agents need a way to pass information. In LangGraph the most common approach is a **shared state**: all agents read from and write to the same state object, so each one can see what the others have done. The alternative is **message passing**, where an agent sends only the specific information the next one needs. Shared state is simpler; message passing keeps each agent's context small and focused, which usually gives better results as systems grow.

Advice before you go multi-agent

It is tempting to reach for many agents because the idea is appealing. In practice, a single well-designed agent is often faster, cheaper and more reliable. Every extra agent adds model calls, delay and opportunities for miscommunication. A good rule: start with one agent, and split only when you can point to a specific problem, such as a prompt that has grown too long or a set of tools that confuse the model, that splitting would solve.

A note on the letters MCP

> ***Worth double-checking on your slide.*** *The slide this guide is based on lists "Multi-agent coordination patterns (MCP)". In the AI world, MCP almost always stands for the* ***Model Context Protocol****, an open standard for connecting models to tools and data sources. It is not an abbreviation for multi-agent coordination patterns. If the slide is going to be presented to others, you may want to change the abbreviation or remove it, so the audience is not confused.*

The two ideas are related but different. Multi-agent patterns are about *how agents organise their work with each other*. The Model Context Protocol is about *how an agent connects to outside tools and information* in a standard way, so that a tool built once can be used by many different applications. In a larger system you can well use both: agents coordinating through a supervisor,

each of them reaching its tools through MCP servers. LangChain and LangGraph have adapters that let agents use tools exposed over MCP.

Part 4: Putting it together

Here is how the three topics on the slide connect. LangChain supplies the components. LangGraph arranges those components into a controllable workflow. The agent lifecycle describes what that workflow does when it runs, and how you grow it from prototype to production. Multi-agent patterns come into play when one workflow is no longer enough and you want several of them to cooperate.

A worked example

Suppose you want a system that produces a short market-research brief on request. Here is one way to build it step by step.

- **Start with one agent.** Give it a search tool and a prompt that says how a good brief looks. Run it on ten sample topics and read the results.
- **Notice the problems.** Perhaps it cites weak sources, or the writing is uneven because it tries to research and write at the same time.
- **Split by job.** Create a researcher agent with search tools and a writer agent with no tools but a strong style prompt. Connect them in LangGraph using a supervisor that sends the topic to the researcher, then passes the notes to the writer.
- **Add a checkpoint for humans.** Pause before the final brief is sent, so a person can approve it.
- **Evaluate again.** Run the same ten topics and compare. Keep the split only if the results improved.
- **Add limits and logging.** Cap the number of steps and record every run, so you can investigate any bad result.

Common mistakes

- Using many agents when one would do.
- Skipping evaluation and relying on a few impressive demos.
- Giving the agent more permissions than the task needs.
- Letting conversations grow without limit until the model loses track of the early instructions.
- Writing vague tool descriptions. The model chooses tools mostly by reading them, so they deserve as much care as a prompt.
- Not setting a stop condition, so the agent can loop indefinitely.

A suggested learning path

1. Make a plain chat model call with LangChain and print the result.
2. Add one tool, and watch the model request it and use the answer.
3. Rebuild the same thing as a LangGraph graph with an agent node, a tools node and a conditional edge.
4. Add a checkpointer so the agent remembers the conversation between turns.
5. Add a human approval step before a sensitive action.
6. Write ten test tasks and score your agent on them.
7. Only then try a supervisor with two specialist agents, and compare against your single-agent baseline.

Quick glossary

Term	Meaning
Agent	A program in which a language model decides what action to take next and repeats until the task is done
Tool	A function the model can request, such as a search or a calculator
State	The shared data a graph carries from step to step
Node / Edge	A step in the graph / the connection that decides which step runs next
Checkpoint	A saved snapshot of state that allows pausing, resuming and replaying
Human-in-the-loop	A design where a person approves or edits at chosen points
Supervisor	An agent that routes work to specialist agents and gathers their results
RAG	Retrieval-augmented generation: fetching relevant documents before the model answers
MCP	Model Context Protocol: an open standard for connecting models to tools and data

Closing thoughts

If you remember only one thing from this guide, let it be this: the model is the easy part. What separates a toy from a dependable system is everything around the model, such as the structure of the workflow, the memory it keeps, the limits it respects and the way you measure its performance. LangGraph gives you a clean way to build that structure, the lifecycle gives you a way to think about it, and multi-agent patterns give you a way to scale it when you need to. Start small, measure honestly, and add complexity only when it earns its place.